

ASSIGNMENT-19.3

Roll no:2303A52074

Batch-37

Task 1 – Python to C++ Conversion

Prompt

Create a simple Python class named Student.

Convert the following Python Student class into C++.

Code

```
class Student:
    def __init__(self, name, age, student_id):
        self.name = name
        self.age = age
        self.student_id = student_id

    def display_student(self):
        print(f"Name: {self.name}, Age: {self.age}, Student ID: {self.student_id}")

# Example of creating a Student object and calling its method
student1 = Student("Alice", 20, "S12345")
student1.display_student()

student2 = Student("Bob", 22, "S67890")
student2.display_student()
```

```
... Name: Alice, Age: 20, Student ID: S12345
... Name: Bob, Age: 22, Student ID: S67890
```

```
%%writefile student.cpp
#include <iostream>
#include <string>

class Student {
public:
    // Constructor
    Student(std::string name, int age, std::string student_id) {
        this->name = name;
        this->age = age;
        this->student_id = student_id;
    }

    // Method to display student information
    void display_student() {
        std::cout << "Name: " << name << ", Age: " << age << ", Student ID: " << student_id << std::endl;
    }

private:
    std::string name;
    int age;
    std::string student_id;
};

int main() {
    // Example of creating Student objects and calling their methods
    Student student1("Alice", 20, "S12345");
    student1.display_student();

    Student student2("Bob", 22, "S67890");
    student2.display_student();

    return 0;
}
```

Output

```
Name: Alice, Age: 20, Student ID: S12345
Name: Bob, Age: 22, Student ID: S67890
```

Explanation

In this task, we created a simple Student class in Python with basic attributes and methods. Then, we used AI to convert the same class into C++, making sure to include constructors, data types, and access specifiers. Finally, we ran both programs and confirmed that the output is the same.

Task 2 – Java to Python Function Conversion

Prompt

Write a simple Java method isPrime().

Convert the following Java isPrime() method into Python.

Code

```
%%writefile PrimeChecker.java

public class PrimeChecker {

    // Method to check if a number is prime
    public static boolean isPrime(int num) {
        if (num <= 1) {
            return false;
        }
        for (int i = 2; i * i <= num; i++) {
            if (num % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println("Is 7 prime? " + isPrime(7));    // Expected: true
        System.out.println("Is 10 prime? " + isPrime(10));  // Expected: false
        System.out.println("Is 2 prime? " + isPrime(2));    // Expected: true
        System.out.println("Is 1 prime? " + isPrime(1));    // Expected: false
        System.out.println("Is 0 prime? " + isPrime(0));    // Expected: false
        System.out.println("Is 29 prime? " + isPrime(29));  // Expected: true
    }
}
```

Writing PrimeChecker.java

, let's compile and run the Java program.

```
def is_prime(num):
    if num <= 1:
        return False
    # We only need to check up to the square root of num
    # In Python, int(num**0.5) gives the floor of the square root
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

# Example usage:
print(f"Is 7 prime? {is_prime(7)}")      # Expected: True
print(f"Is 10 prime? {is_prime(10)}")    # Expected: False
print(f"Is 2 prime? {is_prime(2)}")      # Expected: True
print(f"Is 1 prime? {is_prime(1)}")      # Expected: False
print(f"Is 0 prime? {is_prime(0)}")      # Expected: False
print(f"Is 29 prime? {is_prime(29)}")    # Expected: True
```

Output

```
!javac PrimeChecker.java
!java PrimeChecker
```

```
Is 7 prime? true
Is 10 prime? false
Is 2 prime? true
Is 1 prime? false
Is 0 prime? false
Is 29 prime? true
```

```
... Is 7 prime? True
     Is 10 prime? False
     Is 2 prime? True
     Is 1 prime? False
     Is 0 prime? False
     Is 29 prime? True
```

Explanation

In this task, we took a Java method to check prime numbers and converted it into Python using AI. The logic remained the same, but the syntax was simplified using Python. We tested the function with different inputs to ensure correct results.

Task 3 – Pseudocode to Python Implementation

Prompt

Write pseudocode for Bubble Sort.
Convert the following pseudocode into Python code

Code

```
```\npseudocode\nfunction bubbleSort(list):\n    n = length(list)\n    for i from 0 to n-2:\n        // Last i elements are already in place\n        for j from 0 to n-i-2:\n            // Traverse the list from 0 to n-i-2\n            // Swap if the element found is greater than the next element\n            if list[j] > list[j+1]:\n                swap(list[j], list[j+1])\n    end function\n```\n
```

```
\ndef bubbleSort(arr):\n n = len(arr)\n # Traverse through all array elements\n for i in range(n - 1):\n # Last i elements are already in place\n for j in range(n - i - 1):\n # Traverse the array from 0 to n-i-1\n # Swap if the element found is greater than the next element\n if arr[j] > arr[j + 1]:\n arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap elements\n\n # Example usage:\n my_list = [64, 34, 25, 12, 22, 11, 90]\n\n print("Original list:", my_list)\n bubbleSort(my_list)\n print("Sorted list:", my_list)\n\n my_list_2 = [5, 1, 4, 2, 8]\n print("\nOriginal list 2:", my_list_2)\n bubbleSort(my_list_2)\n print("Sorted list 2:", my_list_2)\n
```

#### Output

```
... Original list: [64, 34, 25, 12, 22, 11, 90]\n Sorted list: [11, 12, 22, 25, 34, 64, 90]\n\n Original list 2: [5, 1, 4, 2, 8]\n Sorted list 2: [1, 2, 4, 5, 8]\n
```

## Explanation

In this task, we started with pseudocode for a sorting algorithm (Bubble Sort). Then, we converted it into Python code using AI. After implementation, we tested it with sample inputs and verified that the list was sorted correctly.

## Task 4 – SQL to Pandas Query

### Prompt

Write a SQL query to select name and salary from employees where salary > 50000. Convert the following SQL query into Pandas code.

### Code

```
SELECT name, salary
FROM employees
WHERE salary > 50000;

import pandas as pd

Create a sample DataFrame to demonstrate (replace with your actual data)
employees_data = {
 'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
 'salary': [60000, 45000, 75000, 50000, 90000]
}
employees_df = pd.DataFrame(employees_data)

Equivalent Pandas code for: SELECT name, salary FROM employees WHERE salary > 50000;
filtered_employees = employees_df[employees_df['salary'] > 50000][['name', 'salary']]
display(filtered_employees)
```

## Output

	name	salary
0	Alice	60000
2	Charlie	75000
4	Eve	90000

## Explanation

In this task, we converted a SQL query into Pandas code. The SQL query filtered employees based on salary, and the same logic was implemented using a DataFrame in Python. The output was tested and matched the expected result.

## Task 5 – Real-Time Application: Algorithm Translation Across

### Prompt

#### Create python code for binary search algorithm

Convert the following Python linear search algorithm into C.

Compare outputs of Python and C programs.

### Code

```
def binary_search(arr, target):
 low = 0
 high = len(arr) - 1

 while low <= high:
 mid = (low + high) // 2
 if arr[mid] == target:
 return mid # Target found at mid index
 elif arr[mid] < target:
 low = mid + 1 # Target is in the right half
 else:
 high = mid - 1 # Target is in the left half
 return -1 # Target not found in the list

Example usage:
sorted_list = [11, 12, 22, 25, 34, 64, 90] # Binary search requires a sorted list
target_1 = 25
target_2 = 100

index_1 = binary_search(sorted_list, target_1)
index_2 = binary_search(sorted_list, target_2)

if index_1 != -1:
 print(f"Element {target_1} found at index {index_1} in the sorted list.")
else:
 print(f"Element {target_1} not found in the sorted list.")

if index_2 != -1:
 print(f"Element {target_2} found at index {index_2} in the sorted list.")
else:
 print(f"Element {target_2} not found in the sorted list.")

Element 25 found at index 3 in the sorted list.
Element 100 not found in the sorted list.
```

```

%%writefile binary_search.c
#include <stdio.h>

// Function to perform binary search
int binarySearch(int arr[], int low, int high, int target) {
 while (low <= high) {
 int mid = low + (high - low) / 2; // To prevent potential overflow

 // Check if target is present at mid
 if (arr[mid] == target) {
 return mid;
 }

 // If target greater, ignore left half
 if (arr[mid] < target) {
 low = mid + 1;
 }
 // If target smaller, ignore right half
 else {
 high = mid - 1;
 }
 }
 return -1; // Target not found
}

int main() {
 int sorted_list[] = {11, 12, 22, 25, 34, 64, 90};
 int n = sizeof(sorted_list) / sizeof(sorted_list[0]);
 int target_1 = 25;
 int target_2 = 100;

 int index_1 = binarySearch(sorted_list, 0, n - 1, target_1);
 int index_2 = binarySearch(sorted_list, 0, n - 1, target_2);
}

```

```

int main() {
 int sorted_list[] = {11, 12, 22, 25, 34, 64, 90};
 int n = sizeof(sorted_list) / sizeof(sorted_list[0]);
 int target_1 = 25;
 int target_2 = 100;

 int index_1 = binarySearch(sorted_list, 0, n - 1, target_1);
 int index_2 = binarySearch(sorted_list, 0, n - 1, target_2);

 if (index_1 != -1) {
 printf("Element %d found at index %d in the sorted list.\n", target_1, index_1);
 } else {
 printf("Element %d not found in the sorted list.\n", target_1);
 }

 if (index_2 != -1) {
 printf("Element %d not found in the sorted list.\n", target_2);
 } else {
 printf("Element %d not found in the sorted list.\n", target_2);
 }

 return 0;
}

```

Writing binary\_search.c

 !gcc binary\_search.c -o binary\_search && ./binary\_search



## Output

```
Element 25 found at index 3 in the sorted list.
Element 100 not found in the sorted list.
```

```
'''Comparing the outputs of the Python and C binary search programs:
...
Python Binary Search Output (from Cell `726d53c3`):
...
Element 25 found at index 3 in the sorted list.
Element 100 not found in the sorted list.
...

C Binary Search Output (from Cell `6ba5ea19`):
...
Element 25 found at index 3 in the sorted list.
Element 100 not found in the sorted list.
...

Both the Python and C implementations of binary search produce identical outputs for the provided test cases. This demonstrates that both programs correctly identify the presence and absence of elements in a sorted list.'''
```

## Explanation

In this task, we selected a Python linear search algorithm and converted it into C language using AI. We executed both versions with the same input and verified that the outputs matched. We also observed differences like syntax, data types, and memory handling.